UNIT 9 BASICS OF SYSTEM DESIGN

9.0 INTRODUCTION

Object Oriented Analysis and Design (OOAD) is an evolutionary development in the field of Software Engineering. The approach taken in object oriented analysis and design(OOAD) differs from the structured analysis and design (SAD) methods, which used algorithm as their fundamental building blocks, and the OOAD harnesses the power of object based and object oriented programming. OOAD uses classes and objects as the basic building blocks. It is a core concept that binds different aspects of system design like interface, database and even computer architecture. The terms Object Oriented Analysis (OOA), Object Oriented Design (OOD) and Object Oriented Programming (OOP) in the OOAD life cycle are iteratively linked to each other. The product of object oriented analysis serves as the basic model from which we can begin the process of object oriented design. The product of object oriented design can be used as the layout for implementing a system using object oriented programming methods. Object-oriented programming is crucial to utilize the benefit of OOAD; otherwise, programmers will not be able to harness the power of OOAD outputs.

In this unit, we will discuss the basics of object oriented design and the steps involved in designing. In this unit, we will also learn how systems are broken into subsystems, concurrency identification, management of data store and boundary conditions.

9.1 OBJECTIVES

After going through this unit, you should be able to :

- Explain the object oriented design,
- Comprehend the need to break down the system into subsystems,
- Describe concurrency and its implementation during design,
- Understand and identify persistent objects and data stores,
- Describe control events,and
- Describe boundary conditions in system design.

9.2 OOA TO OOD

During the object-oriented analysis phase of the software development life cycle, the system designer builds a model that covers each point stated in the requirement specification document. This document is prepared during the requirement analysis phase after rigorous and iterative interactions between the analysis team, the end-user, and the client.

The analysis model is built iteratively, looking into the refinement of the system at each stage, and it is incrementally upgraded to finally build an analysis model which is ready for design and implementation. These iterations are necessary before the analysis model is converged towards the design and implementation of the system. Otherwise a discrepancy left during the analysis phase may lead to faulty design and its implementation, resulting in a huge loss in terms of time and resources.

Once the analysis phase is complete, and models are stable. The analysis models are reviewed by the analysis team and the client to make sure that the system is complete, correct, and consistent with the functional requirements.

The system analyst should ask certain questions to ensure the completeness of the model:

1. Whether each object is used in some use cases. Also, check where these objects are created, modified and deleted in some use cases.
2. Whether each attribute of an objects is used and their types are identified.
3. Whether each association exist in the multiplicity as mentioned i.e., one-to many and many-to-many.
4. Whether each of the control objects in the system has required association with the other objects, which are participating in the use case.

The system analyst should ask certain questions to ensure the correctness of the model:

1. Whether each class is understandable by the user and the client.
2. Whether each description follows the user requirement specifications.
3. Whether each entity and boundary objects have meaningful noun phrases as names.
4. Whether each use case and control objects have meaningful action/verb phrases as names.

The system analyst should ask certain questions to ensure the consistency of the model:

1. Whether each class is named uniquely and used with same name across the use cases.
2. Whether classes having similar attributes present in generalization hierarchy.

The final output of Object Oriented Analysis phase are primarily three main models i.e., object model, functional model, and design model.

- Object Model — This model depicts the system in the form of objects and classes and their attributes and functions. It also explains the aggregation and generalization of classes used in the system.
- Functional Model — This model depicts the system's functionality by using use cases and scenarios. It also explains the system's interaction with the external and internal actors involved with the system.

- Dynamic Model — This model depicts the system's dynamic behaviour by using state charts and sequence diagrams. The state chart diagram depicts the state change in the object when an external event is triggered or internal action is initiated. The sequence diagram shows the interaction between objects and classes.

All these models are used in system design to refine the object model. The object model is designed from the user's perspective in the analysis phase. The object model is refined in the design phase with an eye on its implementation and focusing on actual software components or reusable packages. The system designer also examines the boundary conditions and non-functional requirements that are needed by the system.

For example, classes such as backend database, component, subsystems, networking, and session management do not appear in the analysis object model as the user is completely unaware that these components will become a crucial part of the system.

Objects and classes show a higher level of abstraction during the analysis phase, whereas complete details are needed for its implementation. Hence object model during design exposes all its attributes and features and their interaction with other classes. The boundary classes are also explored, and iteratively new features are added at each refinement stage.

Consider the example of Online Shopping System (OSS) as discussed in the earlier chapter. The OSS website sells products under various categories viz., apparel, food, beverages, toiletries etc. Two types of users interact with the system: web user/customer and administrator. The object model prepared during the analysis phases comprises objects/classes viz., WEB USER, ADMINISTRATOR, PRODUCT GALLERY, PURCHASE, SHIPMENT and DELIVERY. These classes are designed from the user perspective, hence ignoring the system's implementation aspects.

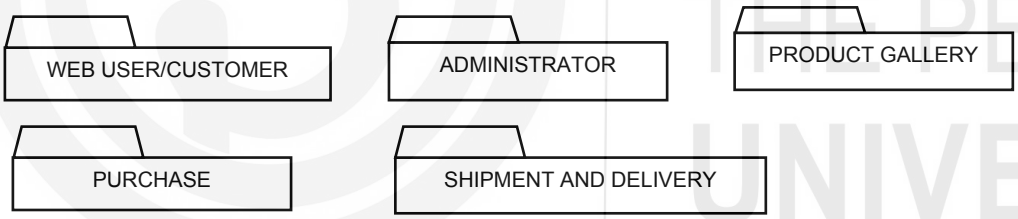

Figure 9.1: Object/Classes identified during System Analysis Phase for Online Shopping System

During the system design phase, the object model is refined, and the new objects/classes are added with an eye on their actual implementation. The primary classes that are ignored are the data storage classes for persistent data. In OSS, the new classes viz., ADMINISTRATOR DATA, WEB USER/CUSTOMER DATA, PRODUCT GALLERY DATA, PURCHASE/ORDER DATA, SHIPMENT AND DELIVERY DATA for storage of these persistent objects are included in the System Design Object Model.

In OSS, customers can make online payment for their purchases. In order to handle the monetary transaction, the system needs to collaborate with various banks through a payment gateway. Moreover, all online communication needs to be secured using security protocol over the Internet. Hence, two more classes, viz., PAYMENT GATEWAY and ONLINE DATA SECURITY needs to be included in the object model.

In OSS, the users are working over internet, the system needs to maintain the session with each customer and in case of inactivity from the customer side, the session may expire. So, we need another class, ONLINE SESSION MANAGEMENT class, to maintain the session of each customer.

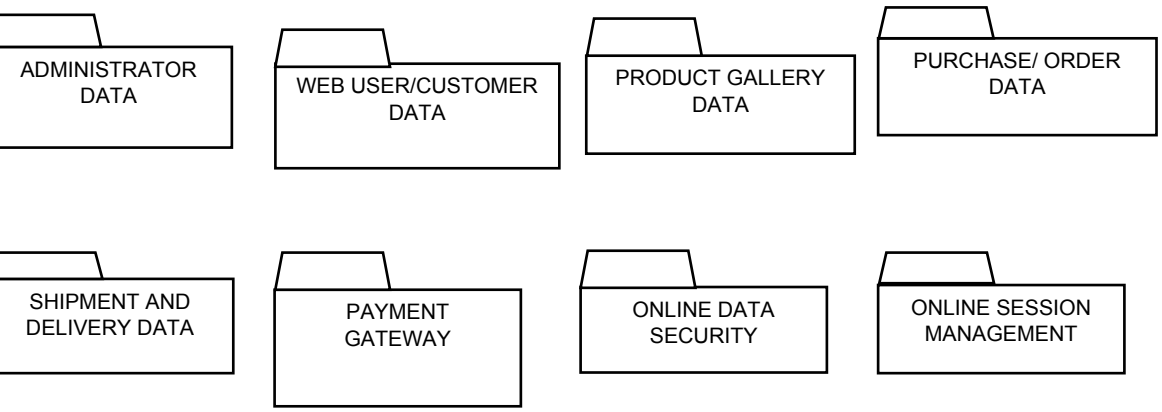

Figure 9.2: Objects/Classes identified during System Design Phases for Online Shopping System

During the system design, eight new classes are added in the OSS object model after looking at the system's non-functional requirements.

9.3 SYSTEM DESIGN: AN OBJECT ORIENTED APPROACH

During system design, the system designer transforms the analysis object model into the design object model by using functional and dynamic models, which explain the functionality and interaction among objects. The system designer formulates the design objectives of the system by looking into the non-functional requirements which must be considered during the implementation of the system.

One of the major steps in system design is decomposition, here the system is decomposed into subsystem in order to reduce the complexity of system and thereby making the smaller subsystems that individual teams can handle. Figure 9.3 shows the relationship of system design with other software engineering activities.

The system designers must also decide about the hardware/software interactions needed by the system, management of persistent data of the system choosing between RDBMS/OODBMS required for storing the persistent data, overall system working, security and access control required and methods to work on boundary conditions. During this phase, the system designer does not have a clear idea of the solution domains; hence it is difficult to anticipate the problems that may generate in the actual implementation of the system.

There are primarily three tasks performed during the system design phase.

1. **Laying down the Design Goals**- The system designer needs to identify the design goals that will improve the design of the system and refine the analysis object model.

2. **Decomposition of the System**- The system designer uses the functional and dynamic model developed during the analysis phase as the basis on which they tend to break the system into subsystems. This process is iterative in nature as a designer needs to carefully divide the complex system into more independent subsystems that have less interactions with other subsystems.

3. **Identification of Boundary Condition**- The system designer needs to identify the system initiation, shutting down of the system, and handling the errors in the system.

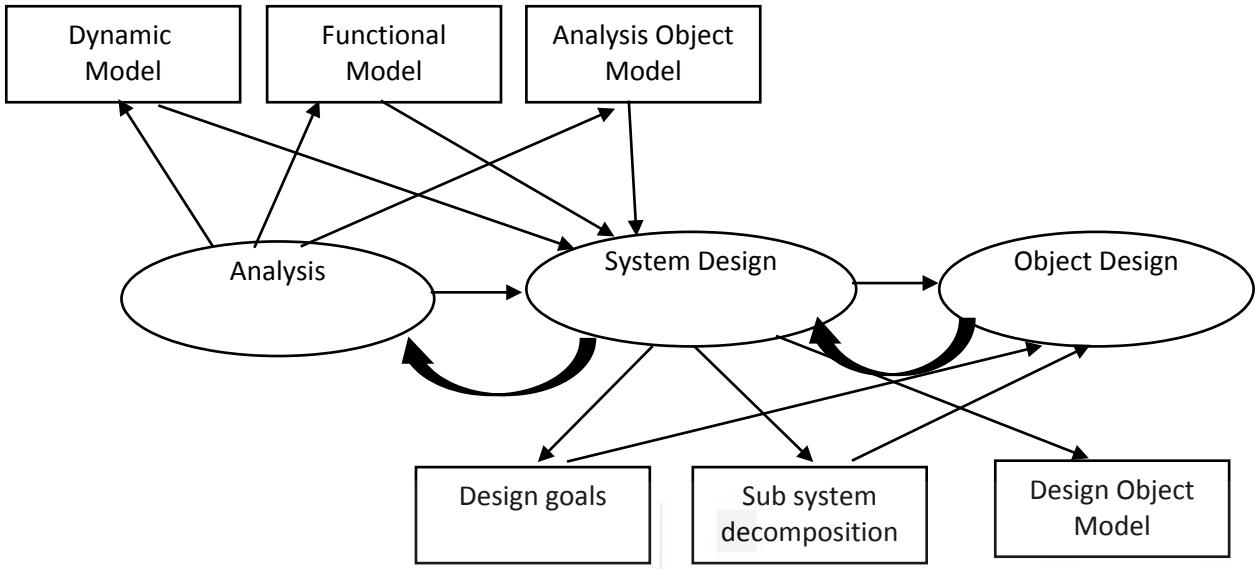

Figure 9.3: The relationship of system design with other software engineering activities.

The system design goals help the system designer to decompose the system into manageable subsystems, reducing the system's complexity and obtaining smaller, manageable independent units. During system decomposition, the system designer needs to focus on certain system-wide design activities:

1) **Reusable components**: The system designer needs to identify the subsystem, which are off-the-shelf components, and the software package available as reusable subsystem. Since these components are already designed and tested, they realize the specific subsystem more economically. The system designer breakdown the complex system into subsystems and identify already available reusable components that can be integrated with the current system. The integration is obtained by designing an interface between the existing component and the new system.
2) **Distributed or networked environment** – When the system is designed for distributed or networked environment, they need an additional subsystem to map the system with hardware infrastructure. And also, the designer needs to address the issues related to networking like authentication, confidentiality, integrity, and durability of data.
3) **Design of persistent data management** – The system designer needs to identify the persistent data that outlives the system execution. This leads to the identification of one or more subsystems to store these persistent data across execution.
4) **Specification of an access control policy in multi-user environment**: In the case of a multiple user system, the user must be given controlled access while dealing with shared classes. There has to be a subsystem that controls the shared access of classes by users and provides them with a level of authentication and control policy for the system.
5) **Design of the system control flow**: The system designer needs to understand the sequence of operations that the end-user will execute. In the process, it identifies the interface required between these subsystems.
6) **Handling the boundary conditions**: Once all the subsystems have been identified and the interfaces have been finalized, the system designer needs to

identify all the external events that will trigger the execution of the system, like system initiation and shutdown etc.

☞ Check your progress-1

Q1. Explain the interaction between the Analysis and Design Phase of OOAD.

Q2. What is System Design?

Q3. What are the major tasks performed during System Design?

Q4. What is persistent data? Why does it need to be taken care of during the design?

9.4 BREAKING INTO SUBSYSTEMS

This section describes the breaking down of complex system into smaller manageable units called subsystem and their properties in more detail.

First, we explain the concept of a system and its relationship with other classes in Object Oriented Software Engineering. Second, we will identify the interaction between the subsystems through interfaces and the services provided by each subsystem.

9.4.1 Subsystems and Classes

In object oriented design, the first step is to simplify the system's complexity by dividing it into smaller subsystems. A subsystem may be defined as a single class or a group of classes that share a common objective and performs multiple services as a single cohesive unit. These subsystems must be independent and must have the least interaction with other subsystems. An individual team or individual developer may handle these subsystems. This results in the concurrent development of an independent unit of work. Finally, these independent subsystems may be combined through interfaces to give a complete working system.

A subsystem may be an off-the-shelf component, third party tool or already existing system in the library. These types of subsystems provide an economical alternative to development from scratch. These components are already successfully deployed in some existing systems, making the system realization simple and cost-effective.

System decomposition is an iterative process, and each design goal contributes to more refinement of a subsystem. A large system is usually decomposed into smaller subsystems using a vertical or horizontal partition. Vertical partition divides a system into several independent (or weakly-coupled) subsystems that provide services on the same level of abstraction. A horizontal partition divides the system into layers, and each layer provides subsystem services to a higher layer (level of abstraction).

Here we have considered an Online Shopping System as an example, it performs three major tasks, and hence the system can be divided vertically into three major subsystems viz., USER MANAGEMENT, PURCHASE MANAGEMENT and PRODUCT GALLERY MANAGEMENT, as shown in Figure 9.4.

Figure 9.4: Vertical Partition of Online Shopping System

The PURCHASE MANAGEMENT subsystem in OSS performs multiple services viz., CUSTOMER LOGIN SESSION MANAGEMENT, SHOPPING CART MANAGEMENT, PURCHASE ORDER, PAYMENT GATEWAY, SHIPMENT AND DELIVERY. Hence, the PURCHASE MANAGEMENT subsystem may further be horizontally partitioned, as shown in Figure 9.5.

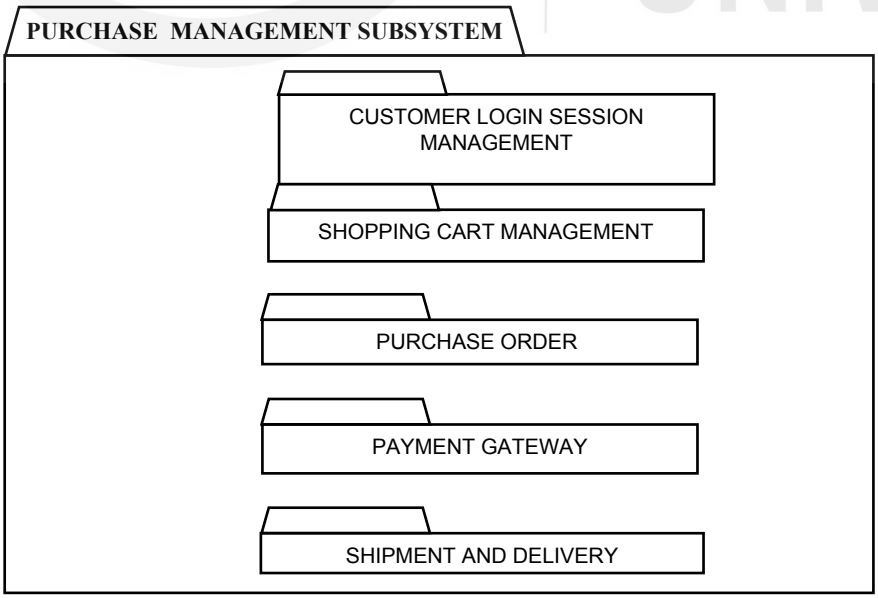

Figure 9.5: Horizontal partition of Purchase Management Subsystem in Online Shopping System

Overall, each subsystem needs an interface to interact with other subsystems. These interfaces are like procedures/functions that accept arguments from other subsystems or users. The subsystem performs the task and returns the output to the calling procedure/function. The interface defines how information flow occurs between the subsystem but does not explain how the subsystem is implemented internally.

In Online Shopping System, we have seen the horizontal partitioning of the PURCHASE MANAGEMENT subsystem into CUSTOMER LOGIN SESSION MANAGEMENT, SHOPPING CART, PURCHASE ORDER, PAYMENT GATEWAY and SHIPPING AND DELIVERY. All these subsystems interact with each other through the interface. CUSTOMER LOGIN SESSION MANAGEMENT subsystem passes the customer login id to the SHOPPING CART MANAGEMENT subsystem, which manages the temporary product selection list of the customer. When a user places the order, the SHOPPING CART system prepares a complete order with product prices picked from PRODUCT GALLERY and passes the details to the PURCHASE ORDER subsystem which after validation passes the total bill to the PAYMENT GATEWAY subsystem. The PAYMENT GATEWAY after successful transaction returns the success message to the SHIPMENT AND DELIVERY subsystem, as shown in Figure 9.6.

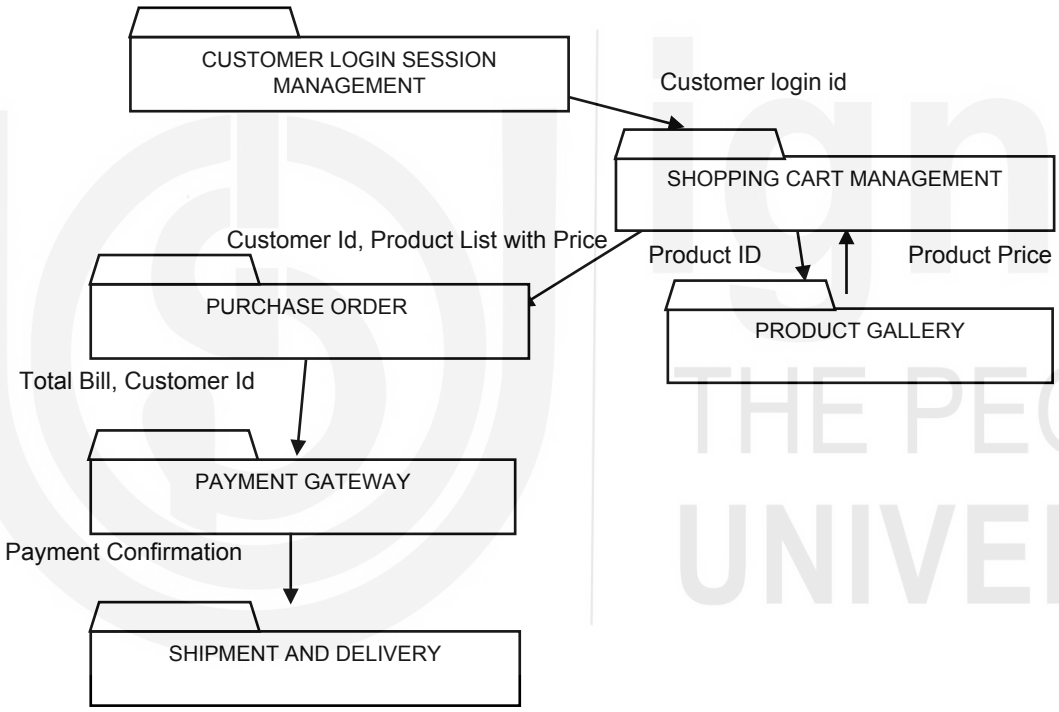

Figure 9.6: The interaction between subsystems of Purchase Management

Consider another real time example of search engine software, it is the most widely used software. The search engine system performs three main tasks: crawling the servers, searching in the databases and presenting the result. The system can be vertically partitioned into three subsystems: CRAWLER, SEARCHER, and PRESENTER as shown in Figure 9.7. The CRAWLER is an independent subsystem of a search engine system, that performs the tasks of crawling the servers across networks, gathering information, and storing the optimized result in the database. The SEARCHER subsystem search for the user query from the database prepared by the CRAWLER subsystem. The PRESENTATION subsystem displays the output result of the SEARCHER subsystem in a sorted manner on the web browser.

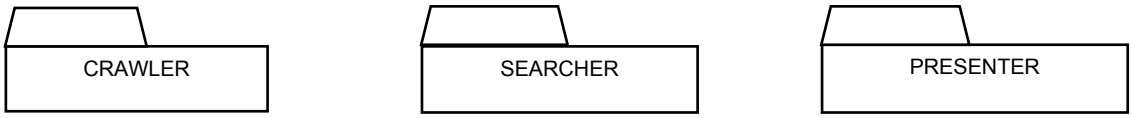

Figure 9.7: Vertical Partitioning of Search Engine System

Each subsystem defines an interface through which they interacts with other subsystems. In this example, the CRAWLER subsystem has no interaction with the SEARCHER subsystem or else PRESENTER subsystems. However, the SEARCHER subsystem is always executed before the PRESENTER subsystem and interacts with the PRESENTER subsystem by passing the search result to the later.

The SEARCHER subsystem performs multiple services; hence it is horizontally partitioned into LEXICON subsystem, INDEX subsystem, RANKER subsystem and SORTING subsystem. Each subsystem performs a predefined task and interacts with each other through the interface, as shown in Figure 9.8.

When user enters the search string, the SEARCHER subsystem processes the query string using the LEXICON subsystem. The LEXICON subsystem parses the query string into keywords, which are further sent to the INDEX subsystem to find the matching web pages list from the DATA SERVER. The INDEX subsystem further passes the matching web pages list to the RANKER and SORTING subsystem, where the web pages are given a ranking based on certain algorithm and sort the result based on the rank. Finally, the ranked web pages list is passed to the PRESENTER subsystem to display the web user's search result.

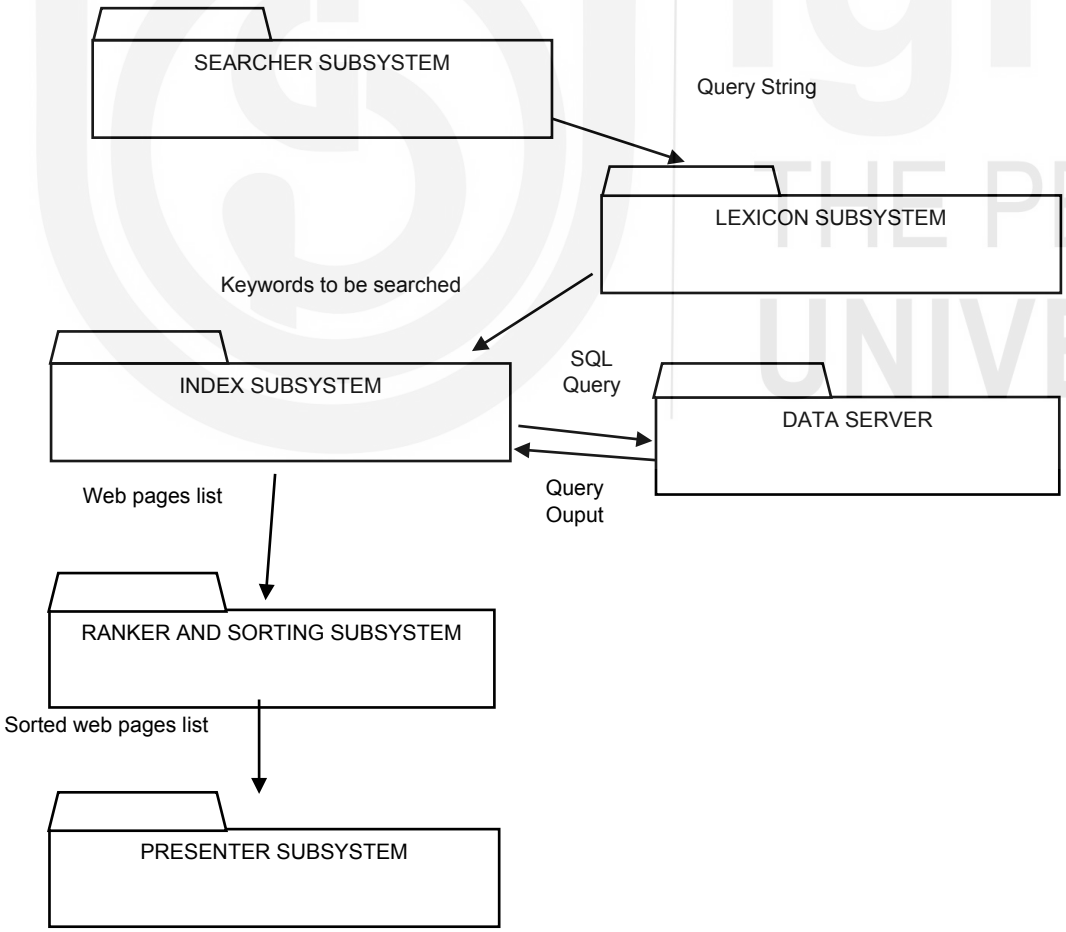

Figure 9.8: Interaction between subsystem in search engine system

9.4.2 Coupling and Cohesion

Two important properties that must be considered while decomposing a system into subsystems, is cohesion and coupling. Cohesion shows the independence of a subsystem, and coupling defines the dependencies between subsystems.

While decomposing a system, the system designer must see whether the subsystem performs all the services on its own or needs little or no services from other subsystems. Such a type of subsystem is called a loosely coupled subsystem. A loosely coupled subsystem is independent and easily manageable. If we make any changes in one subsystem, it will not affect any other subsystem. On the contrary, if a subsystem is highly coupled, then any changes made to a single subsystem will affect all other connected subsystems, thereby stopping other subsystems as well. Hence, it is desirable to have a subsystem that is loosely coupled.

Another important feature of a system is cohesion, which reflects the interdependencies within a subsystem. While decomposing a system, the system designer must strive for subsystems that show high cohesion. A subsystem may have one or more interrelated classes, concurrently performs many related services, and requires little interactions with other subsystems. These types of systems are called cohesive units or independent subsystems. However, if we collaborate classes that are not related to each other and perform unrelated tasks, then such a system cannot be termed cohesive, and it needs to be decomposed into smaller independent subsystems. We tend to decompose a complex system into subsystems and thereby increasing the cohesiveness. But with the increase in the number of subsystems results in an increase in the number of interfaces required to interact with each other. Hence, it results in an increase in coupling due to increase in the number of interfaces.

While decomposing subsystems, there is always a trade-off between cohesion and coupling. If we increase the cohesion, then subsystem may require no information exchange and will work as independent units. If there is a need to reduce the coupling, the system designer needs to provide abstraction to the independent subsystem, which consumes development and processing time.

In Online Shopping System, the CLIENT/WEB USER MANAGEMENT subsystem is a cohesive unit which maintains the details about the customer viz., login details, personal details, bank details etc. and interacts little with other subsystems.

Taking another example from Online Shopping System, consider the PURCHASE ORDER subsystem, as shown in Figure 9.9, there exits high coupling and cohesion between the classes of PURCHASE ORDER subsystem.

The PURCHASE ORDER subsystem internally comprises of six classes, i.e., new order, replacement order, shopping cart, discount redeem, and order details. The order class is generalised into two subclasses, i.e., new order and replacement order. The replacement order class interacts with the order detail class to edit the order details as required by the client. The new order class takes the discount details of the client from the discount redeem class and takes the product list from the shopping cart. The new order class forwards the purchase order details to the order details class. Hence, we can observe that the PURCHASE ORDER subsystem is a cohesive unit, where all the classes interact with one another.

The PURCHASE ORDER subsystem requires client details from the USER MANAGEMENT subsystem. It also requires product details i.e., price, product id, and stock-status from the PRODUCT GALLERY subsystem. And once the order is prepared, the system control moves to the PAYMENT GATEWAY subsystem and waits for the payment confirmation. Finally, the order details are forwarded to the SHIPMENT AND DELIVERY subsystem for delivery to the client when the payment is confirmed.

We observe a high degree of coupling of the PURCHASE ORDER subsystem with other subsystems. All these information's are exchanged through interfaces between these subsystems.

Hence, any changes made to the PURCHASE ORDER subsystem will disrupts the working of PRODUCT GALLERY and SHIPPING AND DELIVERY subsystems. Similarly, any changes made to the SHOPPING CART, DISCOUNT REDEEM or ORDER subsystem will constrain the working of the PURCHASE ORDER subsystem.

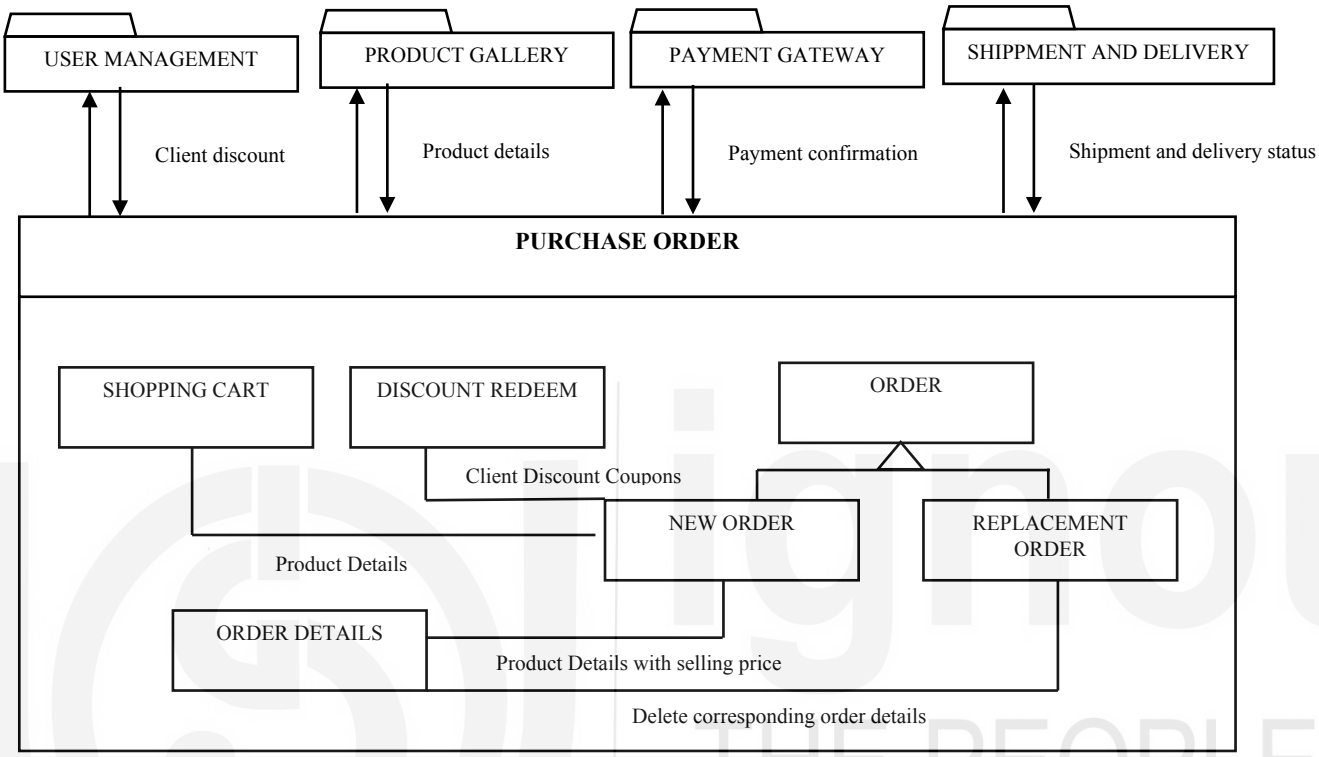

Figure 9.9: Example of PURCHASE ORDER subsystem of Online Shopping System showing cohesion and coupling

In Online Shopping System, in order to store all the data permanently in the relational or object-oriented database we need a separate subsystem for DATABASE MANAGEMENT. We need to design interface for the DATABASE MANAGEMENT subsystem so that other subsystem that needs to store or retrieve data can interact with basic SQL commands.

This results in high coupling between the DATABASE MANAGEMENT subsystem and other subsystems like USER MANAGEMENT, PURCHASE MANAGEMENT and PRODUCT GALLERY MANAGEMENT, as shown in Figure 9.10. Suppose any change is made to the database software, data storage process or retrieval. It will result in a change in all these subsystems.

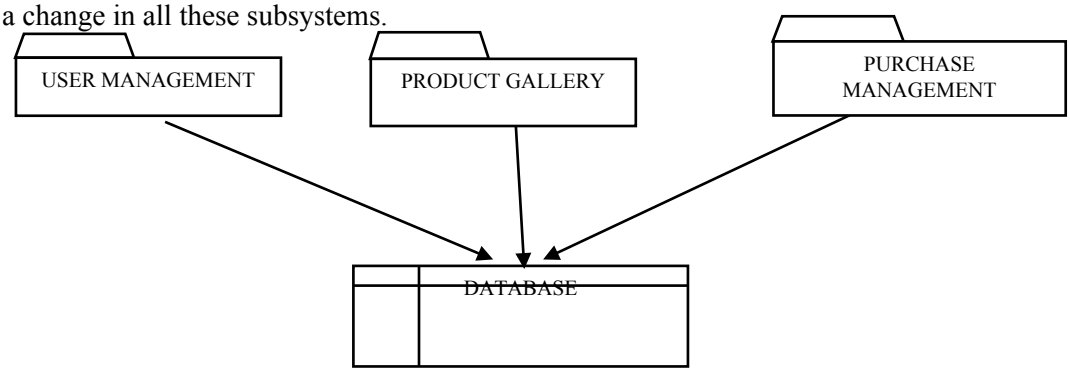

Figure 9.10: Direct access of subsystem to the database

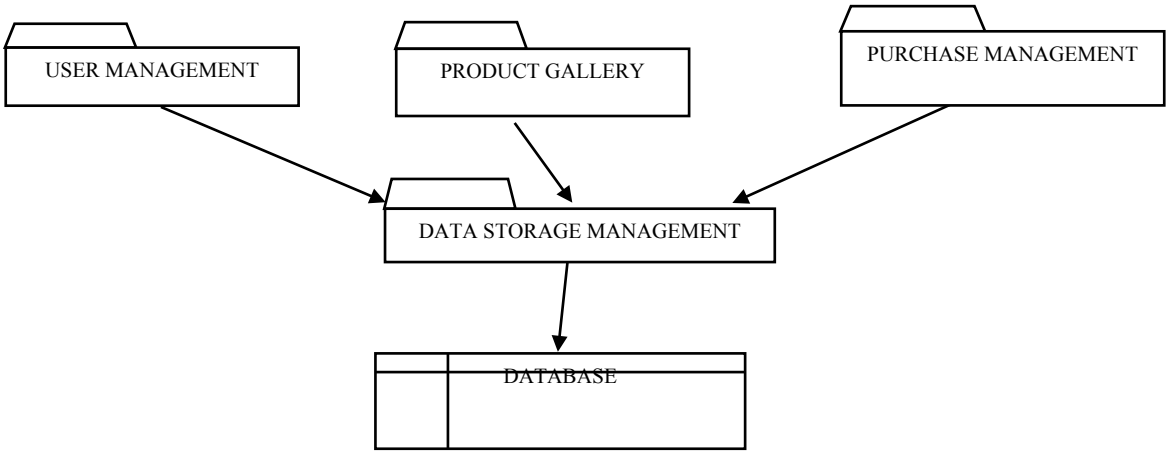

Figure 9.11 : Indirect access of subsystem to the database thorough Data storage subsystem

In order to reduce the coupling between these subsystems, the system designer may decide to create a new subsystem called DATA STORAGE MANAGEMENT, as shown in Figure 9.11, which hides the database functions from all other subsystems. Now, these subsystems can utilize the services of the DATA STORAGE MANAGEMENT subsystem, which is responsible for generating queries for data storage and retrieval from the backend database.

☞ Check your progress-2

Q1. Why do we decompose a system while working with a complex problem?

_____

_____

_____

_____

Q2.What is Cohesion and Coupling?

_____

_____

_____

_____

Q3. Which type of Cohesion and Coupling is required in designing a System?

_____

_____

_____

_____

9.5 CONCURRENCY IDENTIFICATION

Concurrency is the property that distinguishes an active object from inactive object. Concurrency allows two or more objects to be executed at the same time. It allows multiple tasks to execute, interact, and collaborate simultaneously to achieve global functionality of the system.

During design, one must analyse objects that are concurrent in nature and identify mutually exclusive objects.

As in many real-life problems, the system needs to manage different events simultaneously. And also, there are many computationally intensive problems, which exceeds the capacity of a single processor. The possible solution is to use processors capable of multitasking or use a distributed set of computers for implementation.

A single process or task is known as a thread of control, and it is the root from which independent actions are generated within a system. Every executing program has at least one thread of control, but a system that supports concurrency may have more than one threads. Some of these threads are short-lived, whereas others may be active for the lifetime of the system's execution. The systems using concurrency use different scheduling algorithms to execute these threads in parallel. System running on multiple CPUs are purely using concurrency, whereas systems running on a single CPU may not be using full concurrency.

These synchronization algorithms help to avoid situations like

- Deadlock
- Starvation

Deadlock is a condition when two or more threads/processes are holding some resource and waiting for some other resource to be released by another thread or process. This situation is called a deadlock. This results in the creation of a cycle of the hold-n-wait situation, as shown in Figure 9.12. There are two processes P1 and P2, and two resources R1, R2. Process P1 is holding resource R1, which is shown with an arrow directed from resource R1 to process P1. Process P1 is requesting resource R2 as shown with an arrow directed from P1 towards R2. As we can see, the R2 has presently been allocated to process P2, and P2 is requesting resource R1, which is presently being held by process P1 and thus creating a deadlock.

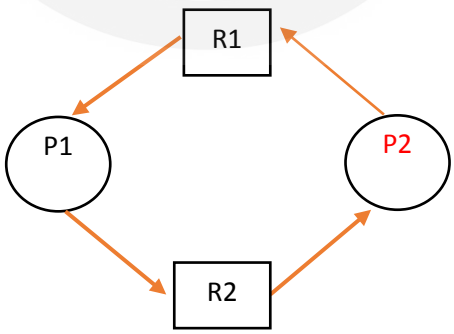

Figure 9.12: Deadlock situation(cyclic hold n wait condition)

Starvation occurs when a thread/process is made to wait for the resource till some other thread/process is using it. But instead of allocating the resource to the waiting thread, the resource is allocated to some other high priority thread/process, resulting in prolonged waiting, leading to starvation.

Concurrency occurs in many multi-user or online systems. The system designer must identify the concurrent objects in the system during system design. These identified concurrent objects need to be implemented diligently in the multi-user environment.

And also, the synchronized access to the shared resources needs to be handled carefully.

Let's take the example of Online Shopping System to explain concurrency, as shown in Figure 9.13. Suppose a web user logs into the system and start surfing through the product gallery. Based on the choice of items selected by the user in the past. The product gallery may be rearranged for increasing the chances of item being selected. At the same time, the online session needs to be maintained. If the user leaves the website untouched for a specific time period, the session may get over, and the system needs to store the temporary shopping cart items in the user's login. All these tasks must happen concurrently and needs to be synchronized.

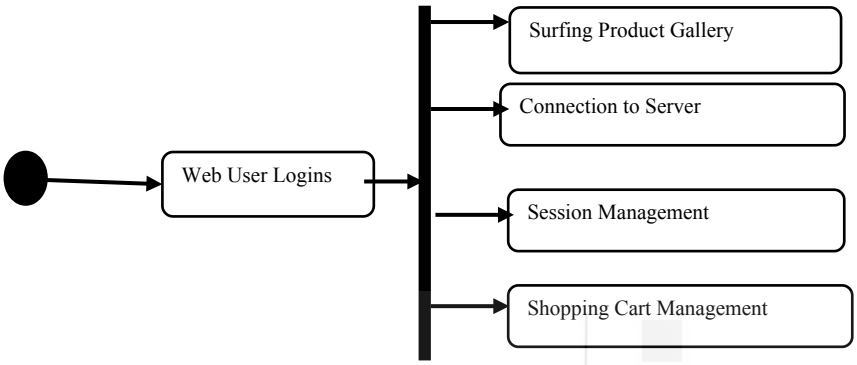

Figure 9.13: Concurrent task in Online Shopping System

Let's consider another example of Mobile Phone System to explain concurrency in a real-time system, as shown in Figure 9.14. Suppose a mobile phone user is editing the contact list. At the same time operating system of the mobile phone is doing lots of tasks viz., connection with the base station (which is the mandatory requirement), running many background activities, and editing the contact list. While at the same time, the mobile user may receive an SMS, and a notification is raised for that SMS. All these tasks must happen concurrently. However, there is sequential ordering in SMS arrival and SMS notification delivery. But there is no ordering in editing the contact list, background processes, and SMS arrival. This means the system is dealing with the concurrent behaviour as different services are running at the same time. Concurrency is a necessary part of the object model and needs to be dealt carefully.

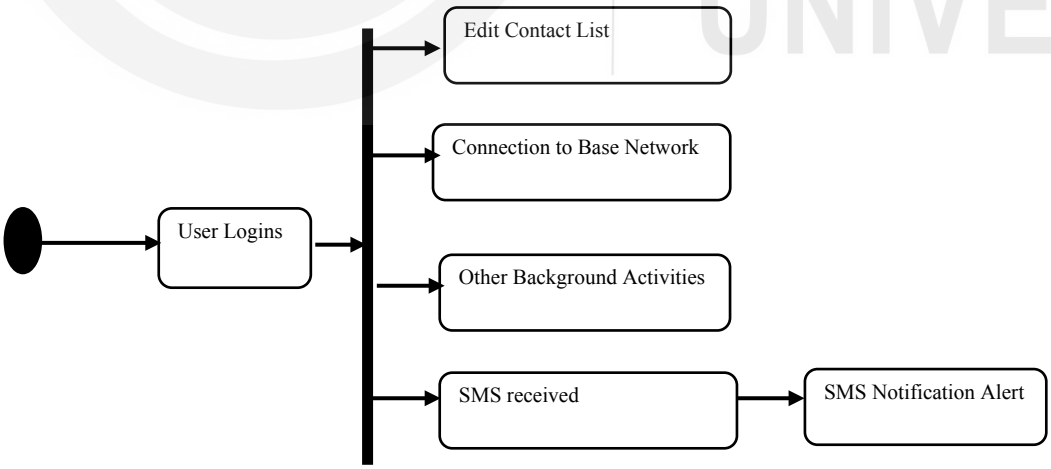

Figure 9.14: Concurrent task in Mobile Phone System

9.6 MANAGEMENT OF A DATA STORE

When we run the system, we tend to create and manipulate the number of objects. What happens to these objects after the system shuts down. Transient objects in the system are deleted after the system shuts down or a process is complete. However, there exist persistent objects that must not be deleted even after the system is shut down, raising the need for databases.

Persistent Objects: The object which must exist even after the system shutdown is called a persistent object. During design, the system designer must identify the persistent object/data that needs to be stored permanently and may be required in future. In the example of an Online Shopping System, we need the Client Information, Order Details, Payment Details, and Product Information for later use and they need to be stored permanently. The permanent storage allows information to be retrieved by the system for later use. It also allows users to run complex queries to retrieve information from multiple files or tables.

The analysis object model serves as a potential candidate to identify persistent data in the system. The system designer needs to separate the classes that must be stored permanently from other temporary objects/classes.

For example, in Online Shopping System, the shopping cart objects need not be stored permanently. They may be stored temporarily in the client system. However, when the user finalizes the order, the shopping cart items must be stored permanently. The user may log in, put the items in the shopping cart, and then leave the website without placing an order. In that case, the shopping cart items need not be stored permanently.

The Data Storage Strategy

Once the persistent objects have been identified, the system designer must decide the storage strategy to be used. The decision for storage is decided by looking at various aspects of the system, viz., non-functional requirements of the user, the complexity of the data that need to be managed, security of data, ease of access required etc.

In general, there are three methods in which persistent data can be stored:

Flat Files: Files are the low-level abstraction provided by the Operating System. They are the simplest method to store persistent objects and provide faster access. The system needs to store a sequence of bytes as flat files. The file system does not support many advanced features of data management, viz., atomicity, consistency, integrity, and durability.

Relational Database: It provides a higher level of data abstraction than flat files. Data is stored in the form of tuples in a relation. The data can be retrieved from multiple tables using complex queries. Relational database exhibits advanced data management features viz., atomicity, consistency, integrity, and durability. They also provide services for concurrency management, access control and recovery.

Object-Oriented Database: These databases provide similar services as relational databases. They follow the properties of OOPS viz., inheritance and abstract data type. They store data in the form of objects and related associations. And also exhibits the highest level of abstraction. The design object model can be directly translated to object databases. However, object-oriented databases are much slower than relational databases for performing queries and are difficult to manage.

Each data storage strategy has its own advantage. It depends on the system being designed to choose amongst the types of the data storage facility.

The advantages of using a file storage system are:

1. The file is one of the simplest data storage mechanisms.

2. It provides easy and fast backup of the data files.
3. It occupies lesser storage space.
4. It is easy to edit any information stored in the files.
5. It provides password security to the files.

The advantages of using relational databases management system are:

1. It provides faster access to the data as compared to file storage system.
2. In the relational database system, there can be multiple tables related to one another with the use of a primary key and foreign key concepts. This makes the data non-repetitive and reduces the chance of inconsistency as in file storage system.
3. It allows concurrent access of data by multiple users by using concurrency management facilities like lock, time stamp and optimistic control.
4. It provides methods for maintaining data integrity, as only authorized users are allowed to access the data.
5. It provides data security.

The advantage of using object-oriented database system are:

1. It stores the data in the form of objects and classes.
2. It allows complex data set to be stored and retrieved much faster.
3. It allows the real world data to be modelled directly into the object oriented database.
4. These databases are capable of storing different types of data viz., pictures, voice, video, text, numbers and so on.
5. The object model can be directly replicated in the object-oriented database.
6. It works well with object-oriented programming languages like C++, Java, VB.NET, C Sharp etc.

RDBMS and OODBMS are the most commonly used database system for storing persistent data. The following Table 9.1 shows the comparison between these systems in terms of certain features:

Table 9.1: Comparison of OODBMS and RDBMS

Features	**OODBMS**	**RDBMS/DBMS**
Definition	An object-oriented database management system is a DBMS where data is represented in the form of objects and classes.	Relational database management systems (RDBMSs), stores data in the form of tables/relations having rows and columns.
Data Type	Object Oriented Database supports complex data, for example, objects of class type.	RDBMS supports basic data types i.e. integer, date, string, Boolean etc.
Data	i) In OODBMS, data are modelled as objects. An object has a finite set of attributes and behavioural properties. ii) Each object has a unique object identifier which is independent	i) In RDBMS, data is modelled as relations. A relation has a finite set of columns or attributes. ii) Each tuple or record in a relation has a unique

	of the values of its attributes. iii) All objects with the same set of attributes and methods are grouped into a class.	identifier called the primary key. iii) Each tuple represents an entity, and a group of similar entities forms an entity set.
Queries	OODBMS supports complex queries, query optimization, backup and recovery.	RDBMS supports very high-level queries, query optimization, transactions, backup and crash recovery.
Object Oriented Features	i)OODBMS supports inheritance, which allows a class to inherit the attributes and methods of a superclass and is represented in the form of a tree. ii) OODBMS also supports multiple inheritance, where a class can have more than one direct superclass and is represented in the form of a lattice. iii) OODBMS also supports encapsulation by grouping attributes and methods under a single class/object. iv) OODBMS supports polymorphism, where a class can define multiple methods with the same name, thereby allowing different objects to respond differently to the same message.	i)RDBMS does not allow the concept of inheritance. ii) RDBMS supports the concept of cardinality in the relationships between entities of the database system.
Query Language	OODBMS supports Object Query Language (OOL). Example, to declare a class in OQL Interface <name> {	RDBMS supports Structured Query language (SQL). Example, to declare relation in SQL Create Table <tablename>

	attributes: <datatype> <name>; relationships <range type> <name>; methods } Method example: float <methodname> (in: <ClassName>)	{ attribute1 datatype, attribute2 datatype, ...}
Security and Access control	OODBMS has only a limited number of security models that have been designed for object oriented data bases. These security models have been specifically designed of object-oriented databases, and this makes use of the concepts of encapsulation, inheritance, and information hiding methods.	RDBMS supports a number of security models primarily categorized as discretionary access control(DAC) policy and mandatory access control(MAC) policy.
Example	ORION, O2, IRIS, POSTGRES	SQL Server, Oracle, MySQL

☞ Check your progress-3

Q1. What do you mean by concurrency?

_____

_____

_____

_____

Q2. Why is the identification of concurrency important in System Design?

_____

_____

_____

_____

Q3. What do you mean by transient and persistent objects?

Q4. What are the methods to store persistent data?

9.7 CONTROLLING EVENTS BETWEEN OBJECTS

The order of execution of a program is called control flow. The flow may be different for different types of systems. In a procedure-oriented system, the program is executed in a sequence, beginning from the main program (start of the program) to the end. However, in an object-oriented system, classes/objects are executed in sequence or concurrently in response to some action or event. The action may be generated by external actors of the system or system's self-triggered events.

During analysis, the flow of a program is not decided, as all the objects/classes are assumed to be executing simultaneously at any time. In the design phase, the system designer must formulate the overall control flow of the system as well as control flow within classes and subsystems. There are many ways in which classes can be called viz., directly by the user action, called by some other class, called by the system itself.

The system designer must choose among several control strategies to implement the control flow of the system. The same style may be implemented throughout the system, or the designer may choose to have a separate control flow for each subsystem. There are usually three types of control flow mechanisms that can be used while implementing the system: procedure-driven, event-driven or thread-based.

Procedure-driven control: In procedure-driven control flow, the procedure waits for the data from the user and then executes the complete program in sequence. When the program is executed, it calls the main function, which waits for the user to enter Login-id and password and then executes the complete program in sequence. The procedure-driven approach is not used nowadays and was used earlier in legacy systems. The example code given in **example Program-1** shows procedure-driven control mechanism and is written in C language.

Example Program-1

```
#include<stdio.h>
```

```
#include<string.h>
void main()
{
char login[15],password[15];
printf("Login Id:");
gets(login);
printf("Password:");
gets(password);
if (!checkpass(login, password))
{
   printf("Invalid login-id or password");
   exit(0);
}
else
   printf("Welcome");
}
int checkpass(char *login, char  *pass)
{
if (strcmp(login,"Admin")==0 && strcmp(pass,"Password")==0)
        return 1;
else
        return 0;
}
```

Event-driven control: In event-driven control, the flow of execution is decided by the action triggered by the user, and then the corresponding piece of code is executed. If users don't perform any action, the system is paused and waits for the user's action.

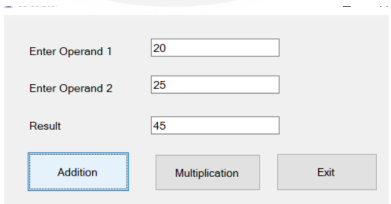

Figure 9.15: An Interface for event –driven system

Example Program-2

```
Public Class Form1
   Dim no1, no2, result As Integer

   Private Sub Button1_Click(sender As System.Object, e As System.EventArgs)
Handles Button1.Click
      'Addition button is clicked
```

```
        no1 = TextBox1.Text
        no2 = TextBox2.Text
        result = no1 + no2
        TextBox3.Text = result
    End Sub

    Private Sub Button2_Click(sender As System.Object, e As System.EventArgs)
Handles Button2.Click
        ‘ Multiplication button is clicked
        no1 = TextBox1.Text
        no2 = TextBox2.Text
        result = no1 * no2
        TextBox3.Text = result
    End Sub

    Private Sub Button3_Click(sender As System.Object, e As System.EventArgs)
Handles Button3.Click
    ‘Exit button is clicked
        End
    End Sub
End Class
```

The example code in example Program-2 shows an event-driven control mechanism written in Microsoft Visual Basic.NET language. The program code is written inside three events, namely Button1_click, Button2_Click and Button3_Click. When the user triggers click action of the button, the corresponding event procedure is executed.

Threads : Threads execute programs concurrently using the concept of parallelism. Threads are an advanced form of program execution than the procedure-driven program. The program can create multiple threads and execute multiple functions concurrently. The execution of a thread is independent of the other threads being executed by the program. The thread provides a very powerful execution mechanism as compared to the other two methods.

Example Program-3

```
class RunThreadDemo implements Runnable

{
  private Thread t;
  private String thrName;

  RunThreadDemo( String tname)
  {
    thrName = tname;
    System.out.println("Creating a Thread" +  thrName );
  }

  public void run()
  {
    System.out.println("Thread Running " +  thrName );
    try
    {
      for(int x = 4; x > 0; x--)
      {
        System.out.println("Thread name: " + thrName + ", " + x);
        // Pausing the thread for a while.
        Thread.sleep(100);
      }
```

```
    }
    catch (InterruptedException e)
    {
      System.out.println("Thread name :" +  thrName + " interrupted.");
    }
    System.out.println("Thread name:" +  thrName + " exits.");
  }

  public void start ()
  {
    System.out.println("Starting " +  thrName );
    if (t == null)
    {
      t = new Thread (this, thrName);
      t.start ();
    }
  }
}

public class TestThread
{
  public static void main(String args[])
  {
   RunThreadDemo R1 = new RunThreadDemo( "Thread-1");
   R1.start();

   RunThreadDemo R2 = new RunThreadDemo ( "Thread-2");
   R2.start();
  }
}
```

In the Example Program-3 , code is written in Java language. It uses the runnable interface to create the class RunThreadDemo. It has two functions: start and run, which create and execute the threads, respectively. The main function creates two threads R1, R2 and shows the execution of two threads in the system.

Each of these control mechanisms has its advantages, and it depends on the system, which control mechanism suits the requirement. The system designer may choose more than one control mechanism in their system.

Once a control flow mechanism is selected, we can realize it with a set of one or more control objects. The role of control objects is to record external events, store temporary states, and issue the right sequence of operation calls on the boundary and entity objects associated with the external event.

9.8 HANDLING BOUNDARY CONDITIONS

In learning how to design systems, we have seen how to decompose the system for better management and implementation. Also, we have seen how to identify and understand the system's data storage requirement and the security measures required by the system. Now, let us see how to identify the boundary conditions of the system like starting, shutdown, etc. We also need to figure out how the system will behave if failure occurs due to error, network failure, database failure, system crash, etc. The use case defined to explain these conditions are called boundary conditions use cases.

In an Online Shopping system, after breaking down the system into subsystems i.e., PURCHASE MANAGEMENT, WEB USER/CUSTOMER, PRODUCT GALLERY. Some common question arises, viz., how a user will start each subsystem, what will happen when a customer transaction fails, or a product update fails or how to manage

multiple customers etc. All these questions need to be answered before implementing the system. Hence, the system designer formulates the boundary conditions use cases for the system.

These boundary conditions are not described during analysis as the complete system is looked upon from the user's perspective. After the initial design, the implementation aspect of the system is far away, as, at the initial design stage, the designer does not bother about how the system will run or behave in case of failure, etc. During the design process, after the design, goals are laid down, and the system is broken down into manageable subsystems, the system designer may come up with different boundary condition use cases by examining each subsystem and objects/classes.

Persistent Objects/Classes - The system designer must examine the use cases where persistent objects are created and destroyed. Suppose some persistent objects are not created or destroyed in the normal working of the system. We need to add a new use case in which the system administrator creates and destroys these objects. Like in Online Shopping System, PRODUCT GALLERY class is accessed in many use cases. But none of the use-case shows the creation and deletion of products in the gallery. Hence, we need a use case where the system administrator adds, modify and delete products in the PRODUCT GALLERY as shown in Figure 9.16.

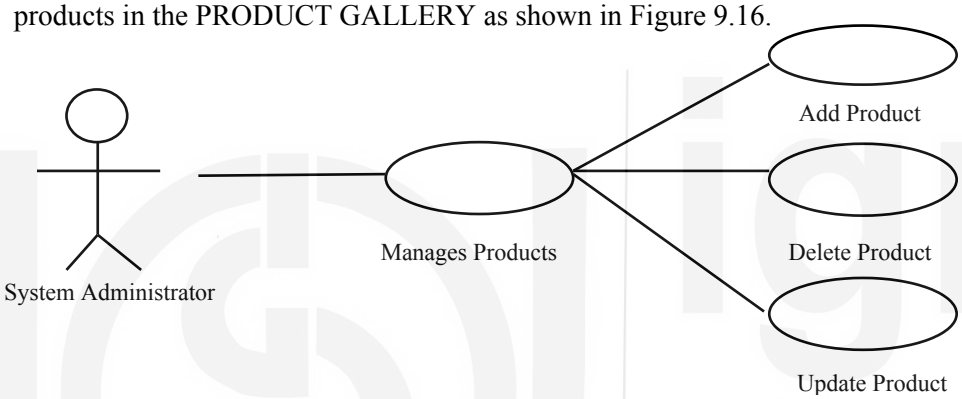

Figure 9.16 : System Administrator use case diagram to manage products in Online Shopping System

Start and Shutdown – The system designer must add three use cases to start, shutdown and configure each subsystem.

Exception Handling – The system designer decides how the system will behave in case of failure of any subsystem. The designer needs to document the handling of each failure condition in the system. For example, in Online Shopping System, the order processing must not start if the user has not selected any product in the shopping cart and triggers the Order processing. In that case, the system must raise an exception and show the relevant message to the user rather than shutting down.

Exceptions are the errors that may disrupt the normal execution of the program. The exceptions are categorized into various categories based on the source of error.

Exception due to hardware failure: A system may crash due to hardware failures like hard disk crash, networking device failure etc. It may lead to permanent data loss if the datastore resides on the same system or it may lead to temporary loss of session data in the case of the online system.

Exception due to software failure: In some cases, the system may shut down due to logical or syntactical errors in programming. Such type of software failure must be handled carefully in the programming.

Exceptions arising due to hardware failures are unavoidable, and hence such situations need to be dealt differently. But software errors must be handled by an appropriate exception handling mechanism. The user must be given an appropriate message if the exception is encountered due to incorrect data entry from the user or incorrect input

from some other function or procedure etc. If any such type of error occurs then appropriate message should be maintained in the system log so that it may be rectified later during system maintenance.

Let's take example of the Online Shopping System. Suppose users enter invalid card details for payment, leading to an error raised by the payment gateway. Such a situation must be handled in such a way that all the purchases listed by the user in the shopping cart are not lost due to an invalid payment error.

One must design a system that should not shutdown abruptly due to any software error or user error. However, an appropriate message must be given to the user in case of any error.

☞ Check your progress-4

Q1. What do you mean by control flow of the system?

_____

_____

_____

_____

Q2. What is the difference between procedure-driven and thread-driven control?

_____

_____

_____

_____

Q3. What are boundary conditions, and how they are dealt?

_____

_____

_____

_____

9.9 SUMMARY

Object Oriented Analysis and Design (OOAD) is an evolutionary development in the field of Software Engineering. It harnesses the power of object-based and object-oriented programming and uses classes and objects as the basic building blocks.

The object oriented analysis phase of the software development life cycle develops three models on the basis of requirement specifications gathered from the user: object model, functional model and dynamic model. The design phase uses these models to further establish the design goals and lay down non-functional requirement before implementation.

During the design phase, a system's complexity is reduced by breaking down the system into smaller independent manageable subsystems. The system designer identifies already existing off-the-shelf components or tools that can be used instead of developing new subsystems from scratch, making the realization of subsystems cost-effective. The next step is to identify the persistent data in the system and lay down a strategy to store the data permanently so that it can be used later in subsequent executions.

The design phase provides the blueprint on which implementation takes place using preferably object oriented programming language. During these steps, many new classes/objects are introduced, thereby refining the object model and making it ready for implementation. These objects need to interact with each other through some control mechanism for their execution. Hence, during design, the system designer chooses the control strategy to be implemented for system execution. Finally, in this unit, the boundary conditions are analyzed looking from the end user perspective for effective start and shut down of the system.

9.10 SOLUTIONS/ANSWERS TO CHECK YOUR PROGRESS

☞ Check your progress-1

1. During the object-oriented analysis phase of the software development life cycle, the system designer builds models viz., object model, functional model and dynamic model, using the requirement specification and these models are built from the user's perspective. The object model is refined in the design phase with an eye on its implementation and focusing on actual software components or reusable packages. Figure 3 explains the interaction between the analysis and design phase in OOAD.
2. System design phase is an iterative process, after system analysis, it refines the analysis model and prepares the blueprint for the implementation of the system. The system designer formulates the design objectives of the system by looking into the non-functional requirements which must be considered during the implementation of the system.

   The major step in system design is decomposition, here, the system is decomposed into subsystems in order to reduce the complexity of the system and thereby to let the smaller subsystems that individual teams can handle. The system designers must also decide about the hardware/software interactions needed by the system, management of persistent data of the system choosing between RDBMS/OODBMS required for storing the persistent data, overall working of the system, security and access control required and methods to work on boundary conditions.
3. There are primarily three major tasks performed during the Design phase.

   1**. Laying down the Design Goals**- The system designer needs to identify the goals that will improve the design of the system and refine the analysis object model.

   2. **Decomposition of System-** The system designer uses the functional and dynamic model developed during the analysis phase as the basis on which they tend to break the system into subsystems. This process is iterative in nature as the designer needs to carefully divide the complex system into more independent subsystems with less interactions with other subsystems.

3. **Identification of boundary condition-** The system designer needs to identify the initiation, shutting down of the system, and handling of the errors in the system.

4. The data which must exist even after the system shutdown is called persistent data. During design, the system designer must identify the persistent data/object that needs to be stored permanently and may be required in future. The system designer needs to identify the persistent data that outlives the system execution, and this leads to the identification of one or more subsystems to store these persistent data across execution.

☞ Check your progress-2

1. To simplify the complexity of the system, it must be divided into smaller sub systems. A subsystem may be defined as a single class or a group of classes that share a common objective and performs multiple services as a single cohesive unit. These subsystems must be independent and must have least interaction with other subsystems.

   An individual team or individual developer may handle these subsystems. This results in the concurrent development of an independent unit of work. Finally, these independent subsystems may be combined through interfaces to give a complete working system.

2. Cohesion shows the independence of a system. It performs all the tasks by itself and requires no or very less interaction with other systems or modules.

   Coupling defines the dependencies between systems. A system may need information from other systems for its work. Hence need to pass and retrieve information through an interface.

3. While decomposing subsystems, there is always a trade-off between cohesion and coupling. A system should have less coupling and high cohesion.

☞ Check your progress-3

1. Concurrency is the property that distinguishes an active object from one that is not active. It allows multiple tasks to execute, interact and collaborate at the same time to achieve global functionality.
2. Concurrency occurs in many of the multi-user or online systems. The system designer must identify the concurrent objects in the system during system design. These identified concurrent objects must be implemented diligently in a multi-user environment. And also, the synchronized access to the shared resources needs to be handled carefully.
3. When we run the system, we tend to create and manipulate a number of objects. Transient objects are deleted after the system shuts down or a process is complete. The persistent objects of these system must not be deleted even after the system is shut down, this raises the need for databases.
4. In general, there are three methods in which persistent data can be stored:

   Flat Files: Files are the low level abstraction provided by the Operating System. The system needs to store a sequence of bytes in the form of flat files.

   *Relational Database***:** It provides a higher level of data abstraction as compared to flat files. Data is stored in the form of tuples in a relation. The data can be retrieved from multiple tables using complex queries.

   *Object Oriented Database***:** These databases provide services similar to relational databases. It stores data in the form of objects and their related associations. It exhibits the highest level of abstraction. The design object model can directly be translated to object databases.

☞ Check your progress-4

1. The order of execution of a program is called control flow. The flow may be different for different types of systems. In a procedure-oriented system, the program is executed in a sequence, from the main program(start of the program) to the end. However, in an object-oriented system, classes/objects are executed in sequence or concurrently in response to some action or event. The action may be generated by external actors of the system or system's self-triggered events.
2. *Procedure-driven control*: In procedure-driven control flow, the procedure waits for the data from the user and then executes the complete program in sequence. The procedure-driven approach is not used nowadays. It was used earlier in legacy systems.

   Thread-driven control: Threads execute programs concurrently to use the concept of parallelism. Threads are an advanced form of program execution than procedure- driven program. The program can create multiple threads and execute multiple functions concurrently. The execution of the thread is independent of the other threads being executed by the program, hence it can wait while others are running. The thread provides a very powerful execution mechanism as compared to other two methods.

3. To identify the boundary conditions of the system like, starting of system, shutdown of system etc. We also need to figure out how the system will behave in case of failure due to error, network failure, database failure, system crash etc. Boundary conditions are identified by looking into subsystems, persistent objects and exceptions. The use case defined to explain these conditions are called boundary conditions use cases.

   Every subsystem needs to be started and shut down; hence system designer makes use-cases of each subsystem.

   Taking an example of an Online shopping system, Figure 9.17 shows the use case of start and shut down of PURCHASE ORDER subsystems by the client/web user

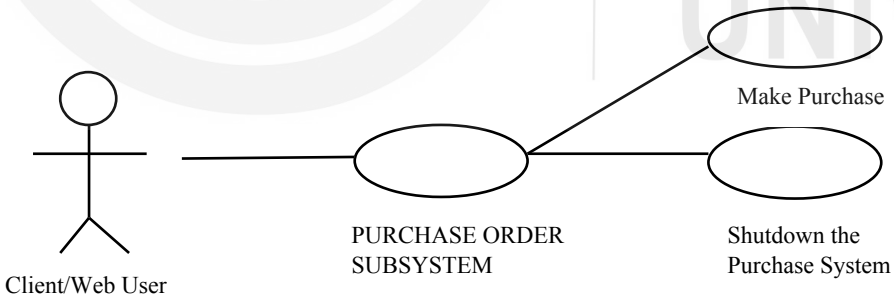

Figure 9.17: Boundary use case for PURCHASE ORDER SUBSYSTEM

Every persistent object needs to be created, modified, and deleted. If it is found missing in the normal working of system use cases. Then special boundary use case is designed to be operated by the system administrator to work on the persistent object.

System designers also need to formulate the exception handling mechanism to be followed in the system. Each type of hardware and software failure needs to be handled elegantly to provide a good working experience to the user.

9.11 REFERENCES/FURTHER READING

- Grady Booch, James Rumbaugh and Ivar Jacobson,“ The Unified Modeling Language User Guide”, 2nd Edition, Addison-Wesley Object Technology Series, 2005.
- Grady Booch, Robert A. Maksimchuk, Michael W. Engle, Bobbi J. Young, Jim Conallen, Kelli A. Houston, “Object-Oriented Analysis and Design with Applications,” 3rd Edition, Addison-Wesley, 2007.
- Grady Booch, Rational, Santa Clara, “Object-Oriented Analysis And Design With applications”, Second Ed., Addison-Wesley.
- James Rambaugh, Michael Blaha, William Premerlam, Frederick Eddy, William Corensen, “Object Oriented Modelling and Design” PHI, New Delhi, 2004.
- Brend Vruegge, Allen H. Dutoit, “Object Oriented Software Engineering Using UML, Patterns, and Java”, Third Ed., Prentice Hall.
- Rebecca Wirfs-Brock, Alan McKean, “Object Design: Roles, Responsibilities, and Collaborations”, Addison Wesley, 2002.
- Ali Bahrami, Irwin , “Object Oriented Systems Development”, Mc Graw-Hill
- Object Oriented Analysis and Design Tutorial, URL: https://www.tutorialspoint.com/object_oriented_analysis_design/index.htm
- Object Oriented Design, URL: https://www.javatpoint.com/software-engineering-object-oriented-design